Identifying LLM fingerprints using machine learning

Fingerprinting LLM families with CNNs and RNNs · CMPSC 448 Midterm Project

Moaz Shalabi

Abstract

A TextCNN I trained can tell which of five LLM families wrote a response 63.5% of the time, against 20% by chance. A BiLSTM with attention gets 60.8%. Simple TF-IDF baselines that read the same 400 tokens score 64.6 to 65.3%, statistically tied with the CNN, so at this data size the neural models do not beat n-gram counts. Most of the signals are stylistic: how a response opens, its punctuation and markdown, and how it lays out its lines. I built a balanced dataset of 13,410 (LLM name, prompt, response) triples from the public Chatbot Arena corpus covering GPT, Claude, Gemini, Llama-2 and Mistral. Ranges in brackets are 95% bootstrap intervals over test prompts.

- RQ1: The CNN reaches 63.5% [61.3, 65.8] and the BiLSTM 60.8% [58.7, 63.0], a 2.7-point gap that holds up. Claude is the easiest family to identify (82 to 89% recall) and Mistral the hardest (42 to 45%).
- RQ2: The prompt alone scores 21.6 to 22.8%, a little above chance but far below the response. Adding the prompt to the response never helps.
- RQ3: Leaving a task type out of training costs the CNN 6.7 points [3.8, 9.7] on that task once difficulty is accounted for. Claude's fingerprint barely changes across tasks.
- RQ4: 25 hand-built style features reach 55% on their own. Shuffling word order costs the CNN 12.7 points, removing punctuation and markdown costs 8.4, and lowercasing has no measurable effect. The first 10 tokens of a response give 49%.

1. Problem definition

I treat LLM attribution as a supervised text classification problem: given a response, and optionally its prompt, predicting which model family produced it. I picked this problem because it has practical uses: spotting undisclosed AI-generated text, checking which provider a product runs on, and seeing how different models still are after training on overlapping data.

Each example is a triple (LLM_name, LLM_input, LLM_output). The label is the model family, meaning the vendor rather than the exact version. Families change less over time than versions do, and labeling by version would split a dataset of this size into too many small classes.

I set out to answer four research questions:

1. RQ1: Can a CNN and an RNN identify which LLM family generates a response?
2. RQ2: How does accuracy change when the model sees only the input, only the output, or both?
3. RQ3: Does a model trained on some task types generalize to task types it never saw?
4. RQ4: Which linguistic or structural features distinguish the families?

2. Dataset curation

The final dataset has 13,410 responses, 2,682 per family, drawn from the public Chatbot Arena human-preference dataset of 57,477 conversations. In Arena, a user writes a prompt, and two randomly chosen models answer it side by side, so each conversation yields two (model, prompt, response) triples, 114,954 in total. The dataset is released on Hugging Face under the Apache-2.0 license.

Family mapping

I kept only first-party models from five vendors so that every label is unambiguous:

Family	Models included
GPT	gpt-3.5-turbo (0314, 0613, 1106, 0125), gpt-4 (0314, 0613, 1106-preview, 0125-preview)
Claude	claude-1, claude-2.0, claude-2.1, claude-instant-1
Gemini	gemini-pro, gemini-pro-dev-api
Llama	llama-2-7b-chat, llama-2-13b-chat, llama-2-70b-chat
Mistral	mistral-7b-instruct (v0.1, v0.2), mistral-medium, mixtral-8x7b-instruct

I left out community fine-tunes such as Vicuna, Zephyr, and OpenHermes. They are built on Llama or Mistral base models and are often trained on GPT outputs, so they have no clear correct label.

Cleaning

I cut multi-turn conversations to their first exchange so that each output depends on exactly one input. I removed null or empty responses, responses under 10 characters, and exact duplicate (model, prompt, response) rows, which left 65,603 usable responses. To balance the classes, I randomly downsampled every family to the size of the smallest, Gemini, using seed 42.

Leakage prevention

I split the data by prompt, not by row: 80% train, 10% validation, 10% test. The script asserts that no prompt appears in more than one split, so a model never sees a test prompt during training. The splits hold 10,705, 1,361, and 1,344 rows, and every family has between 257 and 291 rows in both validation and test.

Splitting by exact prompt does not catch near-duplicates, such as the same request with a few words changed. I compared every test prompt with every training prompt using character n-gram cosine similarity: 80 test rows (6.0%) have a training prompt at 0.9 or above. The CNN is no more accurate on those rows (62.5%) than on the rest (63.6%), so near-duplicates are not inflating the results.

Self-identification

A response that names its maker ("As an AI developed by OpenAI...") would make the task trivial. I flagged responses that mentioned a vendor or model name. Only 0.2 to 0.6% per family were flagged, too few to explain the results, so I kept them.

Personal information

Arena prompts were typed by real users, and the dataset card does not describe any screening for personal information. I did not screen them either. Reading a sample of prompts for the task-label check below, I found a few with full names, one of them attached to a medical-exam request, so the reduced CSV inherits whatever users typed.

Task labels for RQ3

Arena prompts have no task label, so I assigned one with keyword rules applied to the prompt, checked in this order:

Task	Rule (first match wins)	Responses
coding	code fences, or words like *python, function, sql, bug*	1,373
math	words like *solve, integral, probability*, or arithmetic like `3 * 4`	934
creative	words like *write, poem, story, essay, joke*	1,597
qa/other	everything else	9,506

To see how approximate the rules are, A random sample of 40 prompts per label (saved in `extra/task_spotcheck.csv`) was checked. The label was right for 34 of 40 qa/other prompts, 28 of 40 creative, 27 of 40 coding, and only 19 of 40 math, with a few borderline cases in each. Coding picks up role-play games and "Harmonized System code" requests. Math is worse because its arithmetic pattern does off on any two numbers joined by a dash, so ranges, dates, and IDs count as math: 129 of the 806 distinct math prompts matched only that way. Wrong labels dilute the held-out task in RQ3, so the RQ3 drops are probably underestimated, and the math results are the least reliable of the three.

By a rough check (script and English function words), 98% of prompts are in English, so language is not a factor here.

Response length

Mean length ranges from 156 words for Claude to 230 for Llama. Length matters for the neural models because they read only the first 400 tokens: 24% of responses are longer than that, from 6% of Claude's to 35% of Llama's. RQ4 measures how much length contributes as a signal.

3. Models and implementation

I trained two neural models, a TextCNN and a BiLSTM with attention, and compared them with TF-IDF baselines. The neural models are written in PyTorch and the baselines in scikit-learn.

Tokenization

Text is split into words, individual punctuation and markdown characters, and newlines, using the regex `\w+|[^\w\s]|\n`, and case is kept. A standard tokenizer would discard `**`, `#`, bullet dashes, and line breaks, and my hunch was that those formatting habits differ between families. The vocabulary is the 50,000 most frequent training tokens that appear at least twice, with index 0 for padding and 1 for unknown tokens. Sequences are truncated or padded to 400 tokens. In the input + output setting, a sequence is the first 100 prompt tokens, a `[SEP]` token, and then the response.

TextCNN

Following Kim (2014), a 128-dimensional embedding layer feeds three parallel 1-D convolutions with widths 3, 4 and 5 (128 filters each, ReLU). Each convolution is max-pooled over time, the three outputs are concatenated into 384 features, and dropout 0.5 and a linear layer map them to 5 classes. Each filter works as a learned n-gram detector.

BiLSTM with attention

A 128-dimensional embedding feeds a bidirectional LSTM with 128 hidden units per direction, using packed sequences so padding is skipped. Additive attention weights each token and pools the 256-dimensional hidden states into one vector, followed by dropout 0.5 and a linear layer. I used attention pooling because the last hidden state of a long response mostly summarizes its final tokens.

Baselines

Both original baselines use TF-IDF features with sublinear term frequency and multinomial logistic regression ($C = 4$). The word variant uses 1- and 2-grams with min_df 2 and up to 200k features. The character variant uses 2- to 5-grams with min_df 3 and up to 300k features and keeps case.

These baselines differ from the neural models in two ways that favor one side or the other. They read the whole response, while the neural models read only the first 400 tokens. And the word variant uses scikit-learn's default tokenizer, which drops punctuation, markdown, line breaks, and one-letter words, the exact formatting my own tokenizer was built to keep. Section 4 adds versions that remove both differences: a word-level model on the CNN's own tokens, and both baselines cut to the same 400 tokens.

No model's hyperparameters were searched; the code has no tuning loop. The neural settings are standard choices (Kim used the same filter widths and dropout), and $C = 4$ is fixed for every baseline. As a check, choosing C from $\{1, 4, 16\}$ on the validation split moves each new baseline's test accuracy by less than one point.

Training setup

Setting	Value
Optimizer	Adam, learning rate 0.001
Batch size	64
Loss	cross-entropy, unweighted (classes are balanced)
Epochs	up to 15, early stopping after 3 epochs without improvement
Model selection	epoch with the best validation macro-F1
Gradient clipping	max norm 5
Seeds	0, 1 and 2
Hardware	Google Colab, NVIDIA Tesla T4 (16 GB)

Evaluation

I report accuracy and macro-F1 on the test split. With five balanced classes, chance is 20% for both. The test split is small, 1,344 rows from 1,203 prompts, so I give 95% intervals from a bootstrap that resamples test prompts (2,000 resamples; prompts rather than rows, because one prompt can have two responses in the test set). For the neural models I first average each row's correctness over the three seeds. Comparisons between two models are paired: both are scored on the same resampled rows, which gives a tighter interval for the difference than comparing two separate intervals. As a second check on the main comparisons, I use an exact McNemar test on the rows where exactly one of the two models is right.

4. Results

RQ1: Can a model identify which LLM generated a response?

All four models score about three times chance. Neural results are averaged over 3 seeds; the baselines are deterministic.

Model (output only)	Accuracy [95% interval]	Macro-F1
TF-IDF word (1- and 2-grams) + LogReg	0.586 [0.562, 0.612]	0.583
TF-IDF char (2- to 5-grams) + LogReg	0.661 [0.637, 0.687]	0.660
TextCNN	0.635 [0.613, 0.658]	0.636 ± 0.006
BiLSTM + attention	0.608 [0.587, 0.630]	0.609 ± 0.003

The CNN beats the BiLSTM by 2.7 points (paired interval 1.0 to 4.3), and the character baseline beats the CNN by 2.7 points (0.3 to 4.9). Both gaps hold up, but only just: McNemar tests on each seed give p between 0.03 and 0.07 for both. My first draft said the CNN's lead was about nine times the seed-to-seed standard deviation. That compared the gap with the wrong kind of noise. Three

seeds say nothing about how much a different test set would move the numbers, and with 1,344 test rows that alone is about ±2.4 points. The seed spread was also understated, as the note on repeat runs below shows.

The CNN's edge over the BiLSTM suggests the useful signal is local: short phrases and punctuation patterns that a filter of width 3 to 5 picks up directly.

A fairer comparison with the baselines

The baseline numbers in the table are not a like-for-like comparison with the neural models, for the two reasons in Section 3. Figure 1 reruns the baselines on the CNN's tokens and with the same 400-token cut.

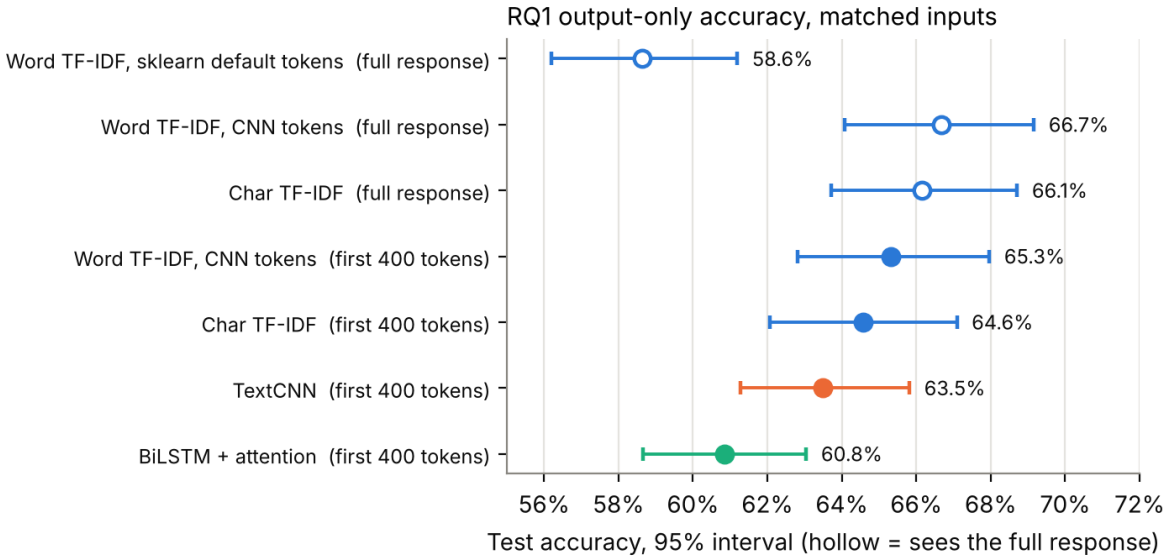


Figure 1. Output-only test accuracy with 95% intervals. Hollow markers read the full response; filled markers read only the first 400 tokens, like the neural models. All baselines use C = 4.

The tokenizer turned out to be the bigger problem. Keeping the tokens the CNN keeps lifts the word baseline from 58.6% to 66.7% (+8.0 points, 5.8 to 10.4), level with the character baseline (66.1%; the difference is 0.5, −1.6 to 2.7). My original explanation, that characters win because they capture sub-word and punctuation patterns, was mostly about punctuation. Once the word model keeps punctuation, words and characters are tied.

Truncation matters less. Cutting the baselines to 400 tokens costs the character model 1.6 points (0.4 to 2.7) and the word-token model 1.3 (0.1 to 2.5). On the 349 test responses that are longer than 400 tokens, where the text is actually missing, it costs about 4 points.

At matched input, the CNN is statistically tied with both baselines. The character baseline at 400 tokens is 1.1 points ahead (−1.3 to 3.5) and the word-token baseline 1.8 ahead (−0.5 to 4.2). If I pick C on the validation set instead of fixing it at 4, the word-token lead grows to 2.6 points (0.3 to 4.9), so the CNN is at best equal to a well-built n-gram model. The BiLSTM stays clearly behind: the 400-token character baseline beats it by 3.7 points (1.4 to 6.2). So my answer to RQ1 is that a CNN can

identify the family about as well as a good n-gram baseline, but at this data size it does not beat one.

Per-family results

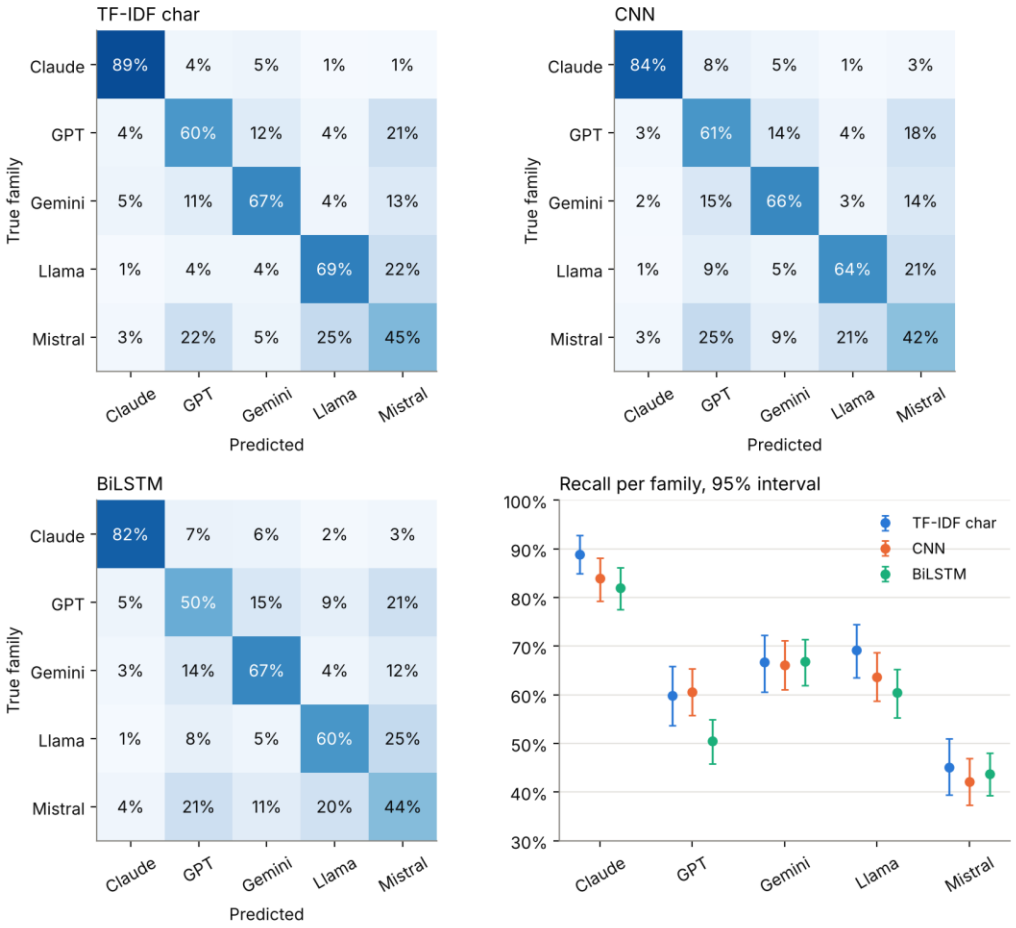


Figure 2. Row-normalized confusion matrices (neural models pooled over 3 seeds) and per-family recall with 95% intervals. The dashed line is chance.

Per-family F1	Claude	GPT	Gemini	Llama	Mistral
TextCNN	0.874	0.560	0.662	0.661	0.423
BiLSTM + attention	0.842	0.504	0.652	0.620	0.425

All three models show the same confusion structure:

- Claude is the most distinct family. It is identified 82 to 89% of the time and rarely confused with the others.
- Mistral is the hardest. It is misread as GPT 21 to 25% of the time and as Llama 20 to 25%, and GPT and Llama responses are often misread as Mistral, so its style sits between the two.

- GPT and Gemini are confused with each other 11 to 15% of the time in each direction.

Each family has only 257 to 291 test rows, so each recall carries an interval of about ±5 points. Claude's lead and Mistral's last place are clear, but the order of GPT, Gemini and Llama in the middle is not.

Two labels mix several models, so I checked whether one easy or hard sub-model drives a family's score. Within GPT, the CNN recognizes GPT-4 responses 63.0% of the time (57.1 to 68.8) and GPT-3.5 responses 56.2% (48.1 to 64.0); the intervals overlap, so I can't say GPT-4 is reliably easier. Within Mistral, the CNN gets 38.6% on Mistral-7B, 45.0% on Mistral-Medium and 41.4% on Mixtral. All three Mistral models are hard to identify.

A note on repeat runs

Fixing the seed did not make GPU training repeatable. The RQ4 "original" condition reruns exactly this CNN configuration with the same three seeds and gets 61.8, 64.2 and 63.0%, against 63.8, 63.2 and 63.5% in the main runs. This is why the ablation results in RQ4 start from 63.0% rather than 63.5%. The likely cause is nondeterministic GPU kernels. Pooled over all six runs, the CNN gets 63.2% with a standard deviation of 0.8 points, more than twice the 0.3 that three seeds suggested.

RQ2: Input only vs. output only vs. both

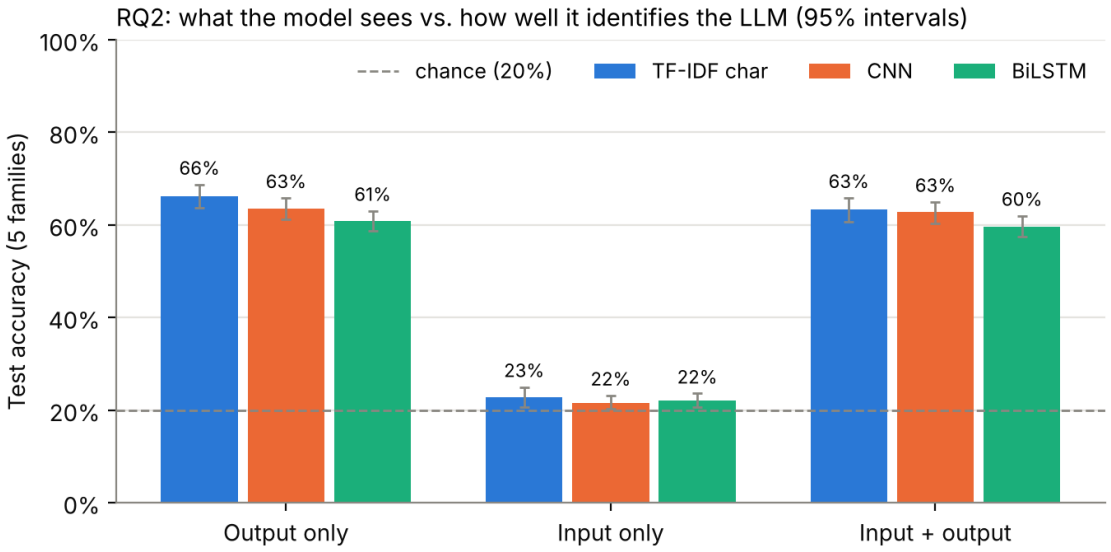


Figure 3. Test accuracy by input setting, with 95% intervals.

Input-only accuracy is 21.6 to 22.8% for all four models. Chatbot Arena assigns two random models to each prompt, so the prompt should not reveal which model answered, and I first called this result chance. That was a little too quick. A permutation test, which shuffles the true labels against each model's predictions 5,000 times, puts chance on this test set at 20.0%, and six of the eight prompt-only runs beat it with p of about 0.01. So the prompt does carry a weak signal of 2 to 3 points. It is small next to the 40-odd points the response adds, so it cannot explain the RQ1 results.

I can only guess where the signal comes from, and I did not test this. My best guess is timing. Models entered the Arena at different times (Claude 2.1, Gemini Pro and Mixtral only from late 2023), and what people ask changes over time, down to prompts that start "Today the date is 02.02.2024". So a prompt can hint at when it was asked, and that hints at which models were available.

Adding the prompt to the response never helps. It costs 0.8 points for the CNN (−0.9 to 2.4) and 1.2 for the BiLSTM (−0.5 to 3.0), neither distinguishable from zero, and 2.8 for the character baseline (1.0 to 4.5). I had blamed part of the neural drop on the prompt using up the 400-token budget. The baselines suggest that is at most part of the story. Cut to the same 400 tokens with the same 100-token prompt, the character baseline still loses 2.2 points when the prompt is added (0.4 to 3.9), though the word-token baseline's loss of 1.1 (−0.7 to 2.9) is within noise. Mostly, the prompt adds tokens that have nothing to do with the label, and the classifier has to learn to ignore them.

5. In-depth analysis

RQ3: Cross-task generalization

For each of coding, creative writing and math, I removed that task from training and validation. I then tested on the regular test rows of the other tasks (in-domain) and on every row of the held-out task (unseen).

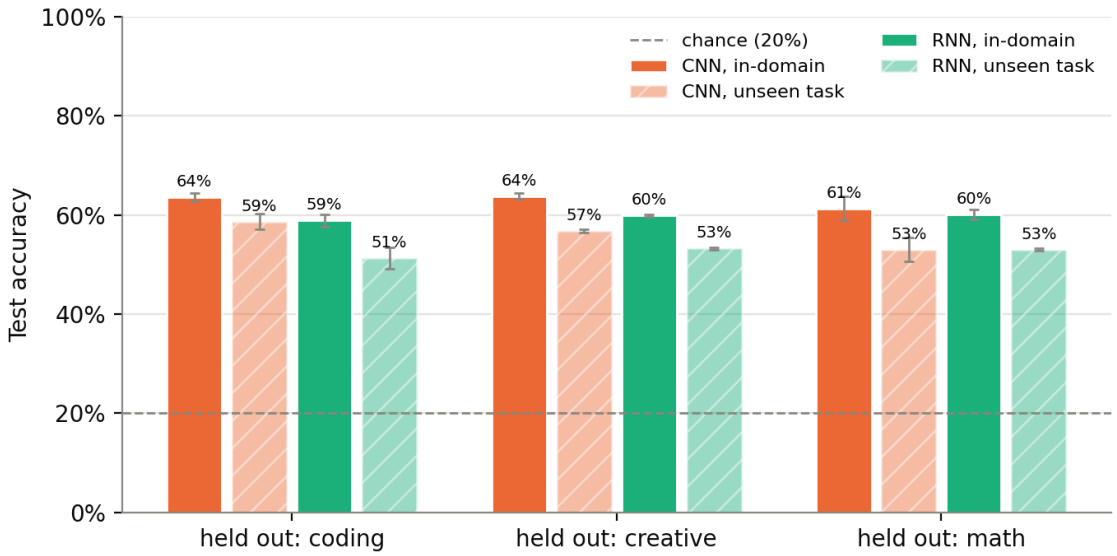


Figure 4. Accuracy in-domain vs. on the held-out task, mean ± std over 3 seeds.

For the CNN the drop is 4.9 points on coding, 7.0 on creative writing and 8.3 on math. The character baseline follows the same order: coding falls from 65.7% to 62.1%, creative writing from 65.8% to 58.6%, and math from 65.6% to 54.6%.

These drops compare different rows, though. In-domain accuracy comes from the other tasks' test rows, and unseen accuracy from the held-out task's rows. If a task is just harder to attribute, a

model would score lower on it even after training on it, and the drop would overstate what failed to transfer. To separate the two, I took the held-out task's rows that are also in the regular test split (157 coding, 174 creative and 87 math rows) and scored them twice. The RQ1 model, which trained on every task, shows how hard the task is. The model that never saw the task, on the same rows, shows what not seeing it costs.

Held-out task (CNN)	Harder than other tasks? (RQ1 model, points)	Cost of never seeing it (same rows, points)
coding (157 rows)	0.7	8.5 [3.6, 13.6]
creative (174 rows)	5.6	5.2 [1.3, 9.5]
math (87 rows)	5.2	6.5 [0.0, 13.2]
all three (418 rows)		6.7 [3.8, 9.7]

Never seeing a task costs the CNN 6.7 points (3.8 to 9.7) pooled over the three tasks, so fingerprints do weaken on a new kind of task. The BiLSTM loses 8.9 (6.0 to 11.9) and the character baseline 4.5 (1.4 to 7.8). The task order I reported in my first draft does not survive. Creative writing and math are about 5 points harder for the CNN even when it trained on them, so a good part of their original drop was difficulty. Coding is no harder than the other tasks, and once difficulty is set aside, its cost is the largest of the three instead of the smallest. With 87 to 174 rows per task the intervals overlap heavily, so I would not rank the tasks at all. My earlier explanation for math, that answers full of equations leave little room for a model's usual openers and formatting, may still be true, but these data cannot show it, especially since only about half the prompts labeled math are really math.

The CNN results by family, pooled over the three held-out tasks and three seeds, with the matched-row cost added:

Family	In-domain accuracy	Unseen-task accuracy	Drop (points)	Cost on matched rows [95%]
Claude	84.6%	84.5%	0.0	2.6 [−0.7, 5.7]
Llama	59.4%	52.1%	7.3	10.5 [4.2, 18.0]
Gemini	63.6%	55.9%	7.7	15.0 [7.4, 23.4]
Mistral	47.1%	38.9%	8.2	2.0 [−5.5, 9.8]
GPT	59.1%	49.5%	9.6	5.0 [−0.9, 11.1]

Claude's stability holds up. It loses 2.6 points on matched rows, with an interval that includes zero. My other claim, that GPT changes its style with the task more than any other family, does not hold. GPT's large raw drop is partly task difficulty; on matched rows it loses 5.0 points (−0.9 to 11.1), and Gemini (15.0) and Llama (10.5) lose more. These per-family numbers rest on 69 to 102 rows each, so the safe conclusion is narrower. Claude writes in the same recognizable way whether it is answering a coding question or writing a poem. For the other four families the fingerprint weakens on a new task, and this test cannot say which family weakens most.

RQ4: What distinguishes the families?

I came at RQ4 from three directions: hand-built features, vocabulary, and retraining the CNN with information removed.

Hand-built style features get most of the way. I computed 25 features per response covering length, sentence length, lexical diversity, markdown use, punctuation rates, first- and second-person rates, stock openers and closers, hedges and refusals. A random forest on these features alone reaches 54.8% accuracy, and logistic regression 45.2%. That is about 83% of the character baseline's 66.1%, from 25 numbers per response.

Feature scaling does not explain the forest's lead over logistic regression. The logistic regression already standardizes its inputs, and log-scaling the skewed counts first only lifts it to 46.9%. The forest is probably using thresholds and combinations of features that a linear model cannot express, but I did not test that directly.

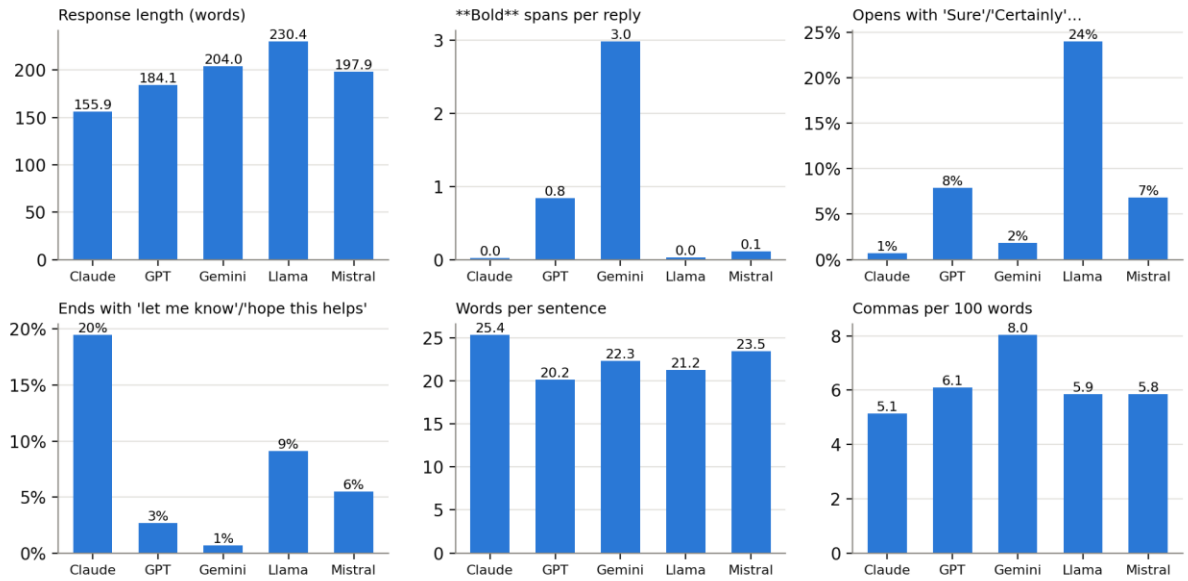


Figure 5. Per-family means of six hand-built features.

I chose the six panels in Figure 5 by looking at the per-family means. That risks showing the features that look most different instead of the ones the classifier relies on. To check, I measured permutation importance: how much the forest's test accuracy drops when one feature's values are shuffled across responses (10 repeats each).

Feature	Accuracy lost (points)	In Figure 5?
lines ending in a colon	10.2	no
bold spans	7.3	yes
length in words	4.6	yes
opens with Sure / Certainly	3.6	yes
type-token ratio, first 100 words	2.2	no
commas per 100 words	2.0	yes
words per line	1.9	no
ends with an offer	1.2	yes
words per sentence	1.2	yes

All six of my panels are in the top nine, so the figure does show features the model uses. But I missed the most important one: lines that end in a colon, like "Here is a summary:" introducing a list. Llama averages about 1.5 such lines per reply and Claude about 1.3, against about 1.0 for the other three families.

Looking at the per-family numbers, each family turned out to have a habit you can spot:

- Gemini uses heavy bold formatting, 3.0 bold spans per reply against 0.8 for GPT and close to zero for the rest. It also uses the most commas, 8.0 per 100 words.
- Llama writes the longest replies (230 words on average) and opens 24% of them with "Sure", "Certainly" or "Of course", three times the rate of the next family.
- Claude writes the shortest replies (156 words) with the longest sentences (25.4 words). It ends 20% of replies with an offer such as "let me know" or "hope this helps", and it uses apology or refusal phrasing most often (13%).
- GPT and Mistral have no single dominant habit among these features, which fits their lower recall in RQ1.

The vocabulary points the same way. My first version of this table used scikit-learn's default tokenizer, which splits "it's" into "it" and a dropped "s" and throws away punctuation. That is how odd entries like "it important" got in. Rebuilt on the CNN's tokens (punctuation, markdown and newlines kept), the highest-weight 1- and 2-grams from a logistic regression show formatting directly. Here `* *` is a bold marker, `\n` a line break, and `m sorry` comes from "I'm sorry":

Family	Most indicative tokens and token pairs
Claude	`Here is`, `\n -`, `So`, `: \n`, `\n •`, `So in`, `summary`, `I do`
GPT	`various`, `As an`, `Remember`, `might`, `* *`, `Certainly !`, `m sorry`, `assist`
Gemini	`* *`, `: *`, `\n -`, `*`, `. *`, `It is`, `a number`
Llama	`Sure`, `Sure !`, `! Here`, `, and`, `: \n`, `Here '`, `just an`
Mistral	`and`, `Note`, `\n Note`, `\ *`, `Note that`, `any questions`

Gemini's list is almost all markdown. Claude's has dash and • bullets and "Here is", Llama opens with "Sure!", and Mistral writes "Note:" and escapes its asterisks as `\*`.

Last, I retrained the CNN (3 seeds each) after removing one kind of information from every response.

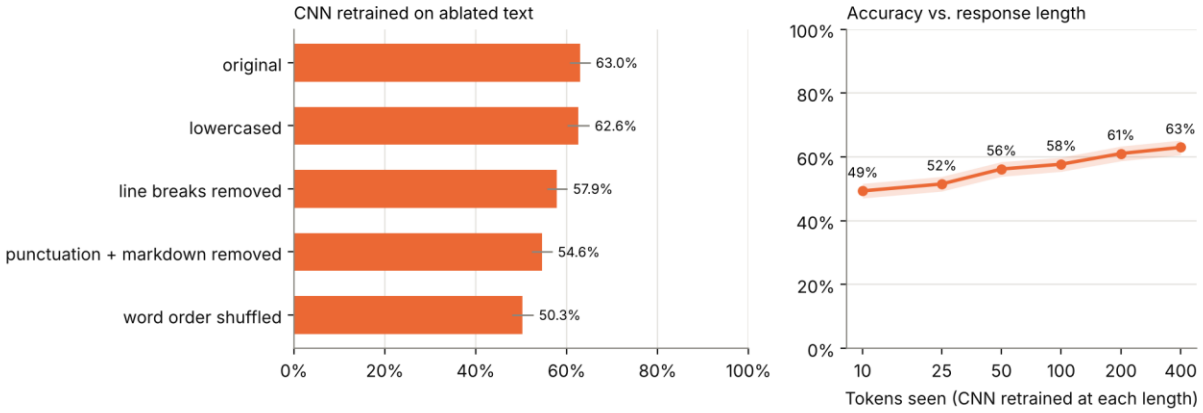


Figure 6. CNN accuracy after retraining on ablated text (left) and on truncated responses (right), with 95% intervals.

Ablation	Accuracy	Change (points) [95%]	What it shows
Original	63.0%	0	reference
Lowercased	62.6%	−0.4 [−1.8, 0.9]	no measurable effect
Line breaks removed	57.9%	−5.1 [−6.8, −3.4]	layout (lists, paragraphing) matters
Punctuation and markdown removed	54.6%	−8.4 [−10.4, −6.4]	punctuation and markdown carry a large share
Word order shuffled	50.3%	−12.7 [−14.6, −10.7]	word order is the largest single source

Shuffling causes the largest loss, but on its own it does not prove that local phrases are what matter, because shuffling also destroys position (what comes first or last) and layout. The word-token TF-IDF model gives a cleaner test, since a bag of n-grams has no notion of position at all. Restricting it to single tokens instead of 1- and 2-grams costs 7.7 points (5.5 to 10.0), and shuffling tokens before counting 2-grams costs 6.9 (4.7 to 9.0). For a model that cannot see position, that loss can only come from adjacent pairs like "Sure !", "Here is" or `**`. So short-range word order carries a lot of the signal, and position adds some more on top, as the next tests show.

One caveat on the punctuation ablation: removing punctuation and markdown also makes each response shorter in tokens, so more of a long response fits under the 400-token cap. If anything, the 8.4-point loss is an underestimate.

The length curve shows that most of the signal sits at the start of a response. Each point is a separate CNN retrained on responses cut to that length, not one model tested on shorter inputs. The first 10 tokens alone give 49.4% accuracy, about 78% of full-length performance. Accuracy

then rises slowly: 56.2% at 50 tokens, 61.1% at 200 and 63.0% at 400. The last step, from 200 to 400 tokens, is small but real (1.9 points, 0.3 to 3.5).

Openings matter more than endings. With the word-token model, the first 10 tokens alone give 50.5% and the last 10 tokens alone 35.6%. Dropping the first 10 tokens from a full response costs 4.1 points (2.3 to 5.8), while dropping the last 10 costs 0.6 (−0.9 to 2.0), within noise. The closing offers in Figure 5 are real habits, especially Claude's, but the classifier leans on how a response starts.

Overall, the signal is both linguistic and structural. Short word sequences, especially at the start of a response, are the largest single source. Formatting (punctuation, markdown and line breaks) is a clear second. Capitalization adds nothing measurable.

6. Lessons learned

The result that surprised me most was simple n-gram baselines matching both neural networks. I expected the CNN and BiLSTM to win because they are more sophisticated, so it was strange to spend so much time building and training them only for a simpler model to keep up. What surprised me even more was why the baseline first looked so strong. Part of its lead came from reading the whole response instead of the first 400 tokens, and the word baseline only looked weak because its tokenizer threw away the formatting. Once every model saw the same input, the CNN and the baselines were tied. The 49.4% accuracy from just the first 10 tokens also stood out. I assumed the classifier would need much more context, but a model's opening phrases already give away a lot of its style.

If I did this project again, I would try pretrained embeddings or a small pretrained transformer before training neural models from scratch. My guess is that 10.7k training examples are not enough for the CNN and BiLSTM to learn embeddings that beat n-gram counts, but I did not run the learning curve that would test it (the script is in `extra/neural_extra.py`). I would also rethink the balancing. Downsampling cut 65,603 usable responses to 13,410, which threw away most of the data I had already cleaned just to even out the classes, so next time I would try class weights and see whether the extra examples help. I would also collect more prompt answers by several families, because only 934 prompts were answered by two or more, which limited how many direct comparisons I could make. (The math task also happens to have 934 responses; the two counts are a coincidence.)

The bugs showed me how easily a broken result can look meaningful. At first GPT appeared to use 13.9 emojis per 100 words, which looked like a big stylistic difference, until I checked the data and found two responses made almost entirely of emoji skewing the average. If I had not looked at the per-family statistics, I might have put that number in the report without questioning it. I changed the feature to a yes-or-no indicator. The word-baseline tokenizer was the same lesson in a quieter form: nothing crashed, the numbers looked plausible, and the odd "it important" in my n-gram table was the only visible clue that punctuation was being dropped.

The evaluation design taught me not to trust an accuracy number without knowing how it was produced. Splitting by prompt mattered because the same prompt in both training and test could have made the classifier look better than it was. The biggest change in this revision was adding intervals. My first draft treated three seeds as the measure of uncertainty and called a 2.7-point gap "nine times the standard deviation". With 1,344 test rows, sampling alone moves accuracy by about ±2.4 points, and two of my comparisons only barely survived once I accounted for it. I also learned that a seed does not make GPU training repeatable: rerunning the same configuration moved one seed by 2 points. Controls changed the story in RQ3 too. Comparing a model with itself on the same rows, instead of comparing different rows, turned "GPT adapts its style the most" into "I can't rank the families".

I used AI assistance for Programing, debugging and for reviewing a draft of this report, and it did not remove the need to check the work. When I asked Claude to critique the draft, some of its suspicions were wrong once I checked them against the code and data: the two 934s it flagged as a possible copy error are both correct, the features it thought might be unscaled were already standardized, and the six features it suspected I had cherry-picked for Figure 5 are all among the nine the classifier relies on most. Others held up, like the tokenizer problem and the missing intervals. Either way, every claim had to be checked against the saved results before it went into the report. Overall, this project made me more careful about questioning unexpected results before assuming that a higher score or an interesting pattern means something.

7. Limitations, reproducibility and references

Limitations

- The Arena data covers models from 2023 to early 2024 (GPT-4-turbo, Claude 2.1, Gemini Pro 1.0, Llama 2), so these fingerprints may not hold for current versions.
- Labels are families, not versions, and each family mixes several model generations.
- Task labels come from keyword rules. A spot check found the math label right for only about half of its prompts, so the RQ3 math results are the weakest.
- The test split has 1,344 rows. Differences between models of under about 2 points cannot be detected, and per-family and per-task results carry intervals of roughly ±4 to ±8 points.
- Balancing kept 13,410 of the 65,603 usable responses (20%).
- Only first turns are used, and neural inputs are cut at 400 tokens (about 300 words), which truncates 24% of responses.
- Seeds do not make the GPU runs exactly repeatable; rerunning the same configuration varied by up to 2 points per seed.
- Prompts come from real Arena users and were not screened for personal information, by the dataset authors as far as the card says, or by me.

Reproducibility

All code and results are in the project repository. The steps run in this order:

Step	Command	Where	Output
1	`python prep.py`	CPU	`data/dataset.csv` and dataset statistics
2	`python baseline.py`	CPU, about 20 min	`results/baseline.csv` (RQ1, RQ2 and RQ3 baselines)
3	`python features.py`	CPU, about 1 min	`results/features_*.csv` (RQ4 features and n-grams)
4	`python models.py`	GPU, about 30 min	`results.csv` and `preds/` (RQ1 and RQ2 neural)
5	`python rq34.py`	GPU, about 50 min	`results/neural.csv` (RQ3 and RQ4 neural)
6	`python figures.py`	CPU	`figures/` (original figures and Table 1)
7	`python extra/baseline_fair.py`	CPU, about 15 min	`results/extra_baseline*.csv` (matched baselines)
8	`python extra/stats.py`	CPU, a few minutes	`results/extra_stats.json`, feature importance and n-gram tables
9	`python extra/figures_extra.py`	CPU	Figures 1, 2, 3 and 6 of this report
optional	`python extra/neural_extra.py`	GPU, about 40 min	learning curve and position ablations (not used in this report)

All randomness is seeded: seed 42 for data sampling and the split, seeds 0, 1, and 2 for the neural models, and seed 0 for the bootstrap and permutation tests. Steps 7 to 9 use only saved predictions and CPU baselines, so they reproduce exactly.

References

- Kim, Y. (2014). Convolutional Neural Networks for Sentence Classification. EMNLP.
- Hochreiter, S. & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation* 9(8).
- Bahdanau, D., Cho, K. & Bengio, Y. (2015). Neural Machine Translation by Jointly Learning to Align and Translate. ICLR. (Additive attention.)
- Zheng, L. et al. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. NeurIPS Datasets and Benchmarks.
- LMArena. arena-human-preference-55k, Hugging Face dataset (Apache-2.0).
- Breiman, L. (2001). Random Forests. *Machine Learning* 45. (Permutation importance.)
- Efron, B. & Tibshirani, R. (1993). *An Introduction to the Bootstrap*. Chapman & Hall.

- McNemar, Q. (1947). Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika* 12(2).
- Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *JMLR* 12.
- Paszke, A. et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. NeurIPS.